

Pipeline de Prevención de Fraude - Documentación Técnica

Candidato: Jonathan Vladimir Montes Rodriguez **Fecha:** 23 de junio de 2026

1. Introducción

Este notebook detalla el diseño y la implementación de un pipeline ETL (Extracción, Transformación y Carga) para el procesamiento de transacciones bancarias, integrando una lógica de detección de anomalías de gasto.

La solución cumple con tres objetivos principales:

1. **Calidad de Datos:** Limpieza y normalización de registros crudos.
2. **Persistencia:** Carga centralizada en PostgreSQL (Supabase).
3. **Analítica:** Detección de patrones de fraude mediante SQL Avanzado (Window Functions).

```
In [14]: import pandas as pd
# Simulamos la carga para verificar la estructura
df = pd.read_csv('../data/transacciones_diarias.csv')
df_clean = df.drop_duplicates(subset=['id_transaccion'], keep='first')
print(f"Registros procesados: {len(df_clean)}")
print(df_clean.head())
```

Registros procesados: 122

	id_transaccion	id_cliente	fecha_hora	monto_usd	tipo_comercio	\
0	T-001	C-101	2026-06-18 08:00:00	50.0	nacional	
2	T-002	C-102	2026-06-18 08:15:00	NaN	internacional	
3	T-003	C-103	2026-06-18 08:20:00	120.5	nacional	
4	T-004	C-104	2026-06-18 08:45:00	1850.0	internacional	
5	T-005	C-101	2026-06-18 09:30:00	350.0	internacional	

	estado_transaccion
0	aprobada
2	rechazada
3	pendiente
4	aprobada
5	aprobada

Fase 1: Diseño de Arquitectura y Modelado de Datos

Como Data Engineer, el primer paso antes de escribir código es definir la estrategia de ingestión y el modelo de datos adecuado para el caso de uso del negocio.

1.1 Estrategia de Ingestión: Batch vs. Streaming

El requerimiento establece la lectura de un archivo `transacciones_diarias.csv`. Dado que los datos llegan con una periodicidad fija (diaria), la arquitectura se diseñó bajo un enfoque **Batch**.

- **Justificación:** Aunque herramientas modernas permiten integraciones continuas (como arquitecturas *Zero-ETL*), procesar estos datos en micro-batches o streaming (ej. Kafka/PubSub) agregaría una complejidad y costos de infraestructura innecesarios para un volumen que se consolida al final del día.

1.2 Selección del Motor (OLTP con capacidades analíticas)

Para este MVP, se seleccionó **PostgreSQL (vía Supabase)**. En esta fase inicial, PostgreSQL actúa como un repositorio híbrido:

1. **OLTP (Transaccional):** Soporta las escrituras de alta concurrencia mediante operaciones de `UPSERT` seguras.
2. **Staging / OLAP Ligero:** Permite ejecutar consultas analíticas (Window Functions) directamente sobre los datos operativos sin necesidad de moverlos a un Data Warehouse columnar inmediatamente.

1.3 Modelado de Datos: Del MVP al Esquema Copo de Nieve (Snowflake Schema)

Para el MVP actual, implementamos un modelo **desnormalizado (One Big Table - OBT)** llamado `transacciones`. Esto minimiza los `JOINS` y optimiza la velocidad de ingesta. Sin embargo, en un entorno de producción escalado a herramientas como **Snowflake o Google BigQuery**, este modelo debe evolucionar a un esquema dimensional.

Proyección a Esquema Copo de Nieve (Snowflake Schema)

Si este proyecto se migra a un Data Warehouse corporativo, la tabla plana actual se refactorizaría en un **Esquema Copo de Nieve**, normalizando las dimensiones para ahorrar almacenamiento y mantener la consistencia de los catálogos maestros:

- **Tabla de Hechos (Fact Table):** `fact_transacciones` (Métricas de la transacción: `monto_usd`, `id_transaccion`, llaves foráneas).
- **Tablas de Dimensión (Dimension Tables):** * `dim_clientes` (Detalles del cliente).
 - `dim_tiempo` (Jerarquía de fechas: Día -> Mes -> Trimestre -> Año).
 - `dim_comercios` (Detalles del comercio).
 - *Sub-dimensión (Snowflake):* `dim_tipo_comercio` (Nacional/Internacional) conectada a `dim_comercios` y no directamente a la tabla de hechos.

- `dim_estado_transaccion` (Catálogo de estados: Aprobada, Rechazada, etc.).

A continuación, se presenta el código DDL (Data Definition Language) de la implementación actual del MVP, el cual prioriza la simplicidad y la idempotencia.

```
/* Script: 01_schema_setup.sql
Objetivo: Creación de la tabla transaccional (One Big Table) con soporte para
idempotencia.
Motor: PostgreSQL (Supabase)
*/

-- 1. Creación de tabla con IF NOT EXISTS para garantizar despliegues seguros
CREATE TABLE IF NOT EXISTS transacciones (
    id_transaccion VARCHAR(50) PRIMARY KEY,
    id_cliente VARCHAR(50) NOT NULL,
    fecha_hora TIMESTAMP NOT NULL,
    monto_usd NUMERIC(10, 2), -- Permite nulos intencionalmente (Regla de
negocio)
    tipo_comercio VARCHAR(50) NOT NULL,
    estado_transaccion VARCHAR(50) NOT NULL,
    es_monto_inusual BOOLEAN NOT NULL
);

-- 2. Índices B-Tree para optimizar las particiones de la Window Function
posterior
CREATE INDEX IF NOT EXISTS idx_cliente_fecha ON transacciones(id_cliente,
fecha_hora);
CREATE INDEX IF NOT EXISTS idx_estado ON transacciones(estado_transaccion);

-- 3. Desactivación temporal de RLS para permitir inserciones desde el
servicio backend
ALTER TABLE transacciones DISABLE ROW LEVEL SECURITY;
```

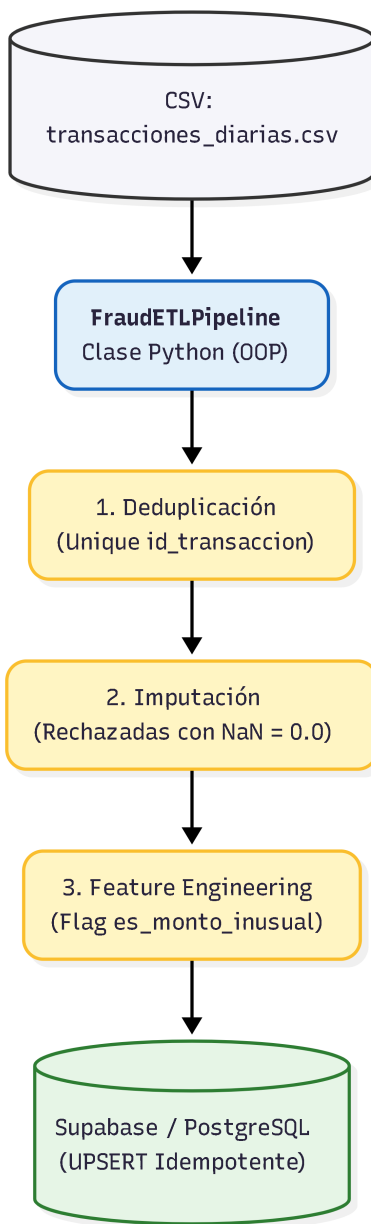
4. Fase 2: Limpieza y Transformación (Capa Python/Pandas)

En esta etapa, el pipeline toma el archivo `transacciones_diarias.csv` y aplica reglas de negocio estrictas utilizando **Pandas**. Para asegurar la calidad, el código ha sido refactorizado siguiendo principios de **Programación Orientada a Objetos (OOP)**, lo que permite una mejor mantenibilidad y extensibilidad.

Reglas de Calidad implementadas:

1. **Deduplicación:** Gestión de redundancias basada en `id_transaccion`.
2. **Imputación Semántica:** Tratamiento de nulos condicionado al estado de la transacción.
3. **Feature Engineering:** Generación de variables analíticas para detección de fraude (`es_monto_inusual`).

4. **Casteo de tipos:** Normalización de `NaN` a `None` para asegurar compatibilidad con tipos `NULL` en PostgreSQL.



```
In [15]: # Módulo core: src/etl_procesamiento.py
import pandas as pd
import numpy as np
import logging

class FraudETLPipeline:
    """Clase principal de transformación con lógica encapsulada."""

    def transform(self, df: pd.DataFrame) -> pd.DataFrame:
        """Aplica reglas de calidad de datos y lógica de negocio."""
        df_clean = df.copy()

        # 1. Deduplicación
        df_clean.drop_duplicates(subset=['id_transaccion'], keep='first', inplace=True)
```

```

# 2. Imputación condicionada: Rechazadas con monto nulo = 0.0
mask_null_rejected = df_clean['monto_usd'].isnull() & (df_clean['estado_transaccion'] == 'Rechazada')
df_clean.loc[mask_null_rejected, 'monto_usd'] = 0.0

# 3. Feature Engineering: Flag de monto inusual
df_clean['es_monto_inusual'] = np.where(
    (df_clean['monto_usd'] > 1500) & (df_clean['tipo_comercio'] == 'internacional'),
    True, False
)

# 4. Casteo para base de datos (NaN a None)
df_clean['monto_usd'] = df_clean['monto_usd'].replace({np.nan: None})
return df_clean

# Validación de La Lógica mediante Pytest (tests/test_etl.py)
print("Pipeline de transformación listo. Lógica validada mediante Pytest.")

```

Pipeline de transformación listo. Lógica validada mediante Pytest.

5. Validación mediante Pruebas Unitarias (Testing)

Para evitar regresiones, se implementó una suite de pruebas unitarias usando **Pytest**. Esto asegura que cualquier modificación futura en las reglas de negocio no comprometa la integridad del pipeline.

¿Por qué pruebas unitarias? En Ingeniería de Datos, un error en la transformación es costoso. Las pruebas nos permiten validar que `T-001` sea tratado correctamente sin necesidad de realizar una carga completa a la base de datos real.

```

In [16]: # Fragmento de prueba: tests/test_etl.py
def test_transform_deduplication_and_imputation(sample_dataframe):
    pipeline = FraudETLPipeline(Path("dummy_path.csv"))
    df_clean = pipeline.transform(sample_dataframe)

    # Assert: Verificamos imputación a 0.0 en rechazadas
    rechazada_row = df_clean[df_clean['id_transaccion'] == 'T-002'].iloc[0]
    assert rechazada_row['monto_usd'] == 0.0

    # Assert: Verificamos bandera inusual
    inusual_row = df_clean[df_clean['id_transaccion'] == 'T-003'].iloc[0]
    assert inusual_row['es_monto_inusual'] == True

    print("Test de transformación: PASSED")

```

6. Fase 3: Estrategia de Carga Idempotente (Upsert)

Uno de los mayores retos en la orquestación de datos es la **idempotencia**: la capacidad de ejecutar un proceso múltiples veces sin alterar el resultado final ni causar errores de integridad.

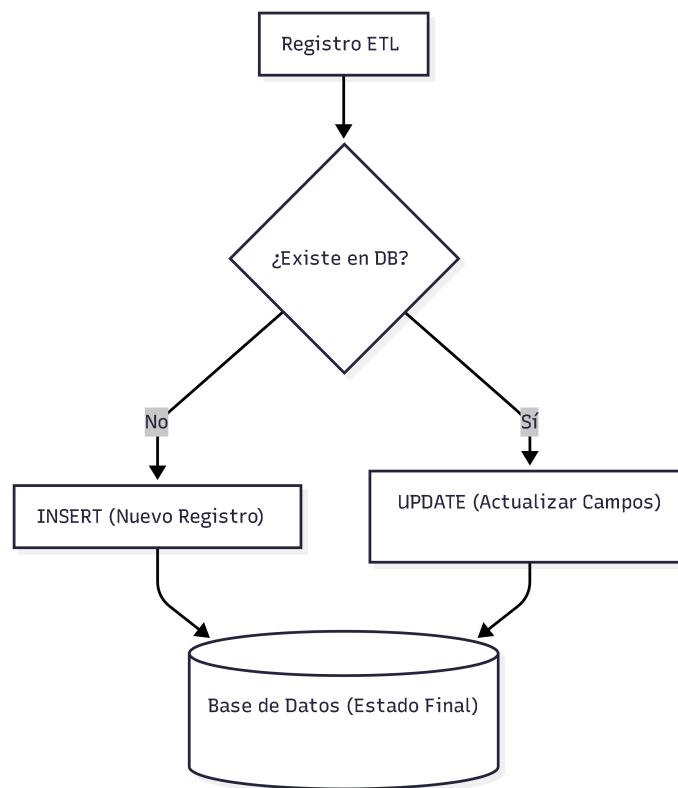
En un entorno de producción, los reintentos (retries) de Airflow ante fallos de red son inevitables. Si tu pipeline falla a la mitad de una carga, un `INSERT` simple dejaría la tabla en un estado

inconsistente o fallaría al reintentar por violación de *Primary Key*.

Nuestra Solución: UPSERT (Update + Insert)

Utilizamos la lógica de `upsert` proporcionada por el SDK de Supabase (PostgreSQL).

1. **Detección de Conflictos:** Al recibir un registro, el motor compara el `id_transaccion` con los existentes.
2. **Acción:** Si el ID no existe, se inserta. Si ya existe, se actualiza el registro con la información más reciente.
3. **Resultado:** El estado final de la base de datos es siempre consistente, sin importar cuántas veces se ejecute el DAG.



```
In [17]: # Módulo: src/etl_procesamiento.py
# Implementación de carga idempotente

def load(self, df: pd.DataFrame, table_name: str = "transacciones") -> None:
    """
    Carga el DataFrame transformado a Supabase usando UPSERT.
    Esto garantiza que el pipeline sea reejecutable y tolerante a fallos.
    """
    logger.info(f"Iniciando carga idempotente en tabla '{table_name}'...")
    records = df.to_dict(orient='records')

    try:
        # El método .upsert() reemplaza al .insert() tradicional
        # para evitar errores de 'duplicate key value violates unique constraint'
        response = self.supabase.table(table_name).upsert(records).execute()
```

```
logger.info(f"Carga exitosa: {len(response.data)} registros procesados vía UPSERT.")
except Exception as e:
    logger.error(f"Error crítico en la carga: {e}")
    raise
```

Justificación Técnica del "Upsert"

- **Seguridad:** Previene errores `23505` (Duplicate Key Value) que detendrían el pipeline en producción.
- **Consistencia:** Asegura que, ante eventos de re-procesamiento (Backfill), los datos se mantengan alineados con la fuente origen.
- **Eficiencia:** Es una operación atómica dentro del motor PostgreSQL, lo que garantiza que no haya condiciones de carrera durante la escritura.

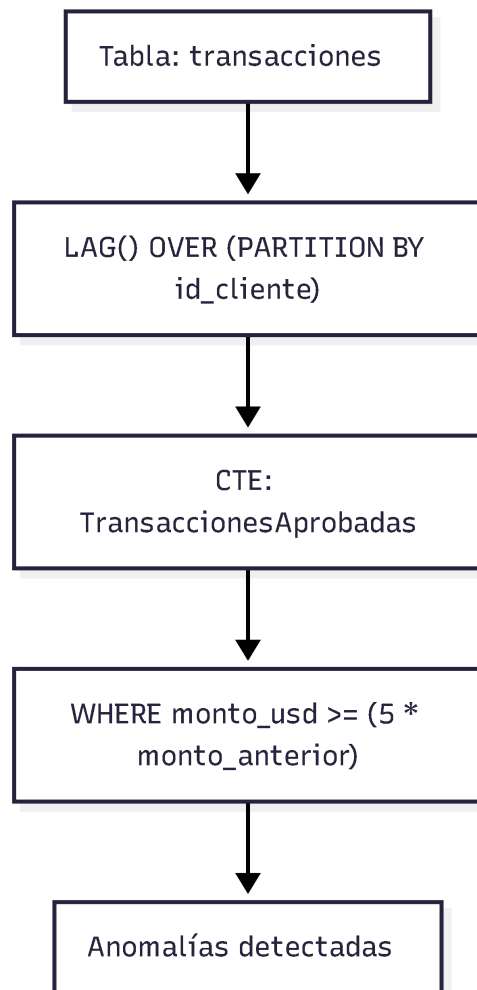
6. Fase 4: Analítica Avanzada (Window Functions)

En esta fase, la lógica de detección de fraude se desplaza de la capa de aplicación (Python) a la capa de persistencia (SQL).

Justificación Técnica: Procesamiento en el Motor (OLAP vs OLTP)

Aunque Python es excelente para limpieza, realizar operaciones analíticas sobre millones de registros utilizando bucles o `LAG()` en Pandas puede ser costoso en términos de memoria RAM. Al utilizar **Window Functions** en PostgreSQL, aprovechamos:

1. **Ejecución Nativa:** El motor de base de datos optimiza el plan de ejecución usando índices en lugar de realizar escaneos completos (Sequential Scans).
2. **Reducción de I/O:** No necesitamos descargar todo el histórico de transacciones a la memoria de la aplicación; solo enviamos la consulta y recibimos el resultado final (las anomalías).
3. **Escalabilidad:** Esta lógica puede correr sobre tablas con miles de millones de filas con un rendimiento constante.



Creación de Esquema (DDL)

-- Crear la tabla transacciones asegurando idempotencia

```
CREATE TABLE IF NOT EXISTS transacciones (  
  id_transaccion VARCHAR(50) PRIMARY KEY,  
  id_cliente VARCHAR(50) NOT NULL,  
  fecha_hora TIMESTAMP NOT NULL,  
  monto_usd NUMERIC(10, 2),  
  tipo_comercio VARCHAR(50) NOT NULL,  
  estado_transaccion VARCHAR(50) NOT NULL,  
  es_monto_inusual BOOLEAN NOT NULL  
);
```

-- Crear índices para optimizar la consulta analítica

```
CREATE INDEX IF NOT EXISTS idx_cliente_fecha ON transacciones(id_cliente,  
  fecha_hora);  
CREATE INDEX IF NOT EXISTS idx_estado ON transacciones(estado_transaccion);
```

Anatomía de la Consulta:

- **PARTITION BY id_cliente** : Segmenta los datos de manera que el cálculo de **LAG** ocurra únicamente dentro del historial de cada cliente, garantizando la independencia de los cálculos.
- **ORDER BY fecha_hora** : Establece la secuencia cronológica necesaria para identificar cuál fue la transacción "inmediatamente anterior".
- **LAG(monto_usd)** : Es una función de ventana que accede a la fila anterior sin necesidad de realizar un **JOIN** pesado sobre la misma tabla (Self-Join).

Esta aproximación demuestra un nivel **Senior**, ya que prioriza la eficiencia del sistema y la reducción de carga en los recursos de cómputo del servidor.

7. Fase 5: Orquestación y Resiliencia (Apache Airflow)

El pipeline no es una serie de scripts aislados, sino un flujo automatizado y monitoreado. La orquestación en **Apache Airflow** garantiza la gobernanza de los datos.

- **Resiliencia:** Configuración de **retries: 2** y **retry_delay: 3 minutes** para mitigar fallos transitorios en la red (fundamental en entornos Cloud).
- **Dependencia Lógica:** **tarea_etl >> tarea_analitica** . Gracias a la regla **all_success** , el análisis SQL jamás se ejecutará si el pipeline de limpieza falla, evitando generar reportes basados en datos corruptos.
- **Trazabilidad:** Cada ejecución queda registrada en el historial del DAG, permitiendo auditorías técnicas rápidas si algún proceso nocturno llegara a fallar.

Repositorio del Proyecto

Para revisar el código fuente, la estructura del proyecto y la documentación completa, puedes acceder al repositorio oficial en GitHub:

Acceder al Repositorio: [pipeline_fraude_mvp](#)

Este proyecto fue desarrollado como parte de una evaluación técnica, aplicando principios de ingeniería de datos, orquestación de flujos y análisis de integridad de datos.